

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К курсовому проектированию
по курсу «Логика и основы алгоритмизации
в инженерных задачах»
На тему «Реализация алгоритма Дейкстры»

Выполнил:
студент группы 24BBB3
Савин А.В.

Приняли:
к.т.н., доцент Юрова О.В.

Пенза 2025

Содержание

Реферат	3
Введение	4
1. Постановка задачи	5
2. Теоретическая часть задания	6
3. Описание алгоритма программы	8
4. Описание программы	10
5. Тестирование	15
6. Ручной расчёт задачи	19
Заключение	21
Список литературы	22
Приложение А. Листинг	23

Реферат

Отчёт содержит 40 страниц, 11 рисунков, 2 таблицы, 6 источников.

ГРАФ, ТЕОРИЯ ГРАФОВ, АЛГОРИТМ ДЕЙКСТРЫ, КРАТЧАЙШИЙ
ПУТЬ, МАТРИЦА СМЕЖНОСТИ, НЕОРИЕНТИРОВАННЫЙ,
ОРИЕНТИРОВАННЫЙ ГРАФ.

Цель исследования - разработка программы, способной находить кратчайшие пути во взвешенном графе между заданной вершиной и всеми остальными вершинами, используя алгоритм Дейкстры.

В работе рассмотрены правила работы алгоритма Дейкстры, на основе которых находится кратчайший путь от стартовой вершины ко всем достижимым вершинам графа. Установлено, что с помощью данного алгоритма можно определить кратчайшие расстояния графа, а также выделить центральные и периферийные вершины.

Введение

Алгоритм Дейкстры является классическим алгоритмом поиска кратчайших путей во взвешенном графе с неотрицательными весами ребер. Разработанный нидерландским ученым Эдсгером Дейкстрой в 1959 году, этот алгоритм находит кратчайшие расстояния от заданной начальной вершины до всех остальных вершин графа.

Отличие алгоритма Дейкстры от других алгоритмов обхода графа (таких как поиск в глубину или поиск в ширину) заключается в том, что он учитывает веса ребер и гарантированно находит оптимальные пути в графах с неотрицательными весами. В то время как поиск в глубину и поиск в ширину работают с ненагруженными графами и находят лишь наличие путей, алгоритм Дейкстры вычисляет минимальные расстояния.

Алгоритм Дейкстры широко применяется в различных областях, включая маршрутизацию в компьютерных сетях, планирование логистических маршрутов, анализ транспортных систем и оптимизацию сетевых структур. Его эффективность и надежность сделали его одним из наиболее важных алгоритмов в информатике и исследовании операций.

Если граф ориентированный, то алгоритм Дейкстры строит дерево кратчайших путей из начальной вершины во все достижимые из нее вершины. Для неориентированных графов алгоритм работает аналогичным образом, учитывая симметричность связей.

В качестве среды разработки была выбрана среда Microsoft Visual Studio, язык программирования – Си, который обеспечивает эффективную работу с динамическими структурами данных и низкоуровневый контроль над памятью, что особенно важно при обработке.

Целью данной курсовой работы является разработка программы на языке Си, который является широко используемым. Именно с его помощью в данном курсовом проекте реализуется алгоритм Дейкстры, осуществляющий поиск кратчайших путей в графе.

1. Постановка задачи

Требуется разработать программу, которая найдет кратчайшие пути от заданной вершины до всех остальных вершин взвешенного графа, используя алгоритм Дейкстры.

Исходный граф в программе должен задаваться матрицей смежности, причём при генерации данных должны быть предусмотрены граничные случаи (разреженные и плотные графы, ориентированные и неориентированные). Программа должна работать так, чтобы пользователь мог ввести или сгенерировать количество вершин и веса рёбер для матрицы смежности. После обработки этих данных на экран должна выводиться матрица смежности графа, тип графа (ориентированный/неориентированный), кратчайшие расстояния от выбранной вершины до всех остальных, а также сами найденные кратчайшие пути.

Необходимо предусмотреть различные исходы поиска (недостижимые вершины, изолированные вершины, циклы), чтобы программа не выдавала ошибок и работала правильно. Устройство ввода – клавиатура и мышь.

Задания выполняются в соответствии с вариантом №13.

2. Теоретическая часть задания

Граф G (рисунок 1) задаётся множеством вершин $X_1, X_2, \dots, X_n$ и множеством рёбер, соединяющих между собой определённые вершины. Рёбра могут иметь веса, которые обозначают "стоимость" перехода между вершинами. Граф может быть ориентированным, где рёбра имеют направление, или неориентированным, где рёбра не имеют направления.

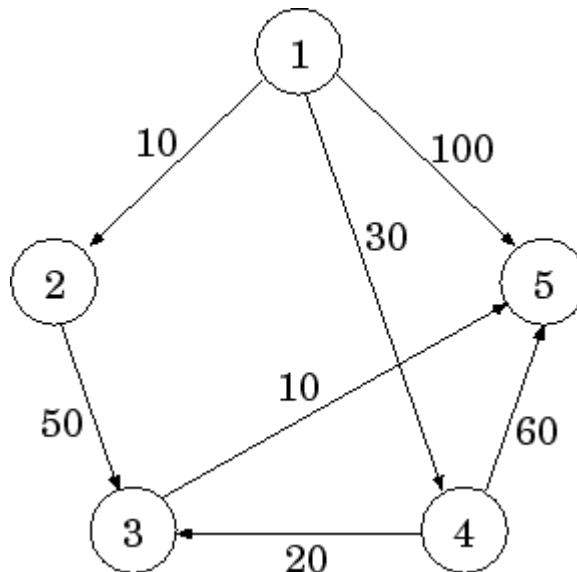


Рисунок 1 – Пример взвешенного графа

При представлении графа матрицей смежности информация о рёбрах графа хранится в квадратной матрице, где наличие пути из одной вершины в другую обозначается весом ребра, а отсутствие пути - нулем.

Существует много алгоритмов на графах, в основе которых лежит поиск оптимальных путей между вершинами. Остановимся на одном из классических методов решения задачи поиска кратчайших путей - алгоритме Дейкстры.

Пусть задан граф $G = (X, A)$ с неотрицательными весами ребер и вершина X_1 , с которой начинается поиск кратчайших путей. Алгоритм Дейкстры последовательно находит кратчайшие расстояния от начальной вершины ко всем остальным вершинам графа. На каждом шаге алгоритм выбирает вершину с наименьшим известным расстоянием от начальной

вершины, помечает её как посещённую и обновляет расстояния до всех её соседей.

Так, продолжая выбирать вершины с минимальными расстояниями, алгоритм находит кратчайшие пути до всех достижимых вершин графа. Алгоритм прекращает свою работу после обработки всех вершин графа или когда все оставшиеся вершины недостижимы.

3. Описание алгоритма программы

Для программной реализации алгоритма понадобятся четыре массива: `dist (int)` - для хранения кратчайших расстояний от начальной вершины, `visited (int)`, в который будут заноситься посещенные вершины (1 или 0), `prev (int)` для хранения предыдущей вершины в пути для восстановления маршрута и `graph` (двумерный массив `int`), в котором хранится матрица смежности графа. Итак, имеется граф $G=(V, E)$. Каждая из вершин входящих во множество V , изначально имеет очень большое расстояние (`INT_MAX`), т.е. элементам массива `dist` присвоено значение `INT_MAX`.

В качестве исходного пункта выбирается вершина s и ей приписывается расстояние 0: `dist[s]=0`. Далее, находится вершина с минимальным расстоянием среди непосещенных вершин и, поочерёдно исследуя её соседей, обновляются расстояния до них. Так как из вершины существует несколько путей, то возможно такое, когда текущий путь оказывается длиннее уже найденного. Поэтому алгоритм на каждом шаге выбирает оптимальную вершину и продолжает поиск из неё.

Далее происходит обработка найденных расстояний. Если расстояние до вершины равно `INT_MAX`, то вершина недостижима из начальной. Для достижимых вершин происходит восстановление пути с помощью массива `prev`, в который на каждом шаге записывалась предыдущая вершина в кратчайшем пути.

Если граф ориентированный, то матрица смежности несимметричная и пути могут быть только в одном направлении. Если граф неориентированный, то матрица симметричная и пути возможны в обоих направлениях.

Алгоритм продолжается до тех пор, пока все кратчайшие пути не будут найдены. Ниже представлен псевдокод функции `dijkstra()`.

dijkstra (начальная_вершина s, матрица graph, n, dist, prev)

1. для $i=0$ пока $i < n$ делать $i=i+1$
 2. $\text{dist}[i] = \text{INT_MAX}$
 3. $\text{visited}[i] = 0$
 4. $\text{prev}[i] = -1$
5. конец цикла
6. $\text{dist}[s] = 0$
7. для $\text{count}=0$ пока $\text{count} < n$ делать $\text{count}=\text{count}+1$
 8. $u = -1$
 9. $\text{min_dist} = \text{INT_MAX}$
 10. для $i=0$ пока $i < n$ делать $i=i+1$
 11. если $\text{visited}[i] == 0$ и $\text{dist}[i] < \text{min_dist}$
 12. $\text{min_dist} = \text{dist}[i]$
 13. $u = i$
 14. конец условия и цикла
 15. если $u == -1$
 16. прервать цикл
 17. $\text{visited}[u] = 1$
 18. для $v=0$ пока $v < n$ делать $v=v+1$
 19. если $\text{visited}[v] == 0$ и $\text{graph}[u][v] > 0$
 20. $\text{new_dist} = \text{dist}[u] + \text{graph}[u][v]$
 21. если $\text{new_dist} < \text{dist}[v]$
 22. $\text{dist}[v] = \text{new_dist}$
 23. $\text{prev}[v] = u$
 24. конец условия и цикла
 25. конец цикла

4. Описание программы

Для написания данной программы использован язык программирования Си. Язык программирования Си - универсальный язык программирования, который завоевал особую популярность у программистов, благодаря сочетанию возможностей языков программирования высокого и низкого уровней.

Проект был создан в виде консольного приложения. Данная программа является многомодульной, поскольку состоит из нескольких функций: `main`, `dijkstra`, `printPath`, `inputManual`, `inputRandom`, `inputFromFile`, `exportGraph`, `printGraph`, и вспомогательных функций для работы с памятью.

Работа программы начинается с вывода меню на экран:

```
printf("1. Задать размер графа (текущий: %d)\n", n);
printf("2. Задать тип графа (текущий: %s)\n",
isDirected ? "ориентированный" : "неориентированный");
printf("3. Ввод графа вручную\n");
printf("4. Генерация случайного графа\n");
printf("5. Загрузить граф из файла\n");
printf("6. Экспорт графа в файл\n");
printf("7. Показать матрицу смежности\n");
printf("8. Выполнить алгоритм Дейкстры\n");
printf("0. Выход\n");
```

Если пользователь выбрал генерацию случайного графа (пункт 4), то программа запрашивает параметры генерации. Программа формирует граф с заданными параметрами. Для каждого возможного ребра (кроме петель) генерируется случайное число от 0 до 99. Если это число меньше значения `density`, то создается ребро со случайным весом от 1 до `maxWeight`. Для неориентированного графа ребра создаются симметрично.

Если пользователь выбрал ввод графа вручную (пункт 3), программа запрашивает элементы матрицы смежности:

```
printf("Введите матрицу смежности (%dx%d):\n", n, n);
printf("(0 - нет ребра, >0 - вес ребра)\n");

for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
if (i == j) {
graph[i][j] = 0;
continue;
}

printf("graph[%d][%d]: ", i, j);
int weight;
scanf("%d", &weight);

if (weight < 0) {
printf("Вес не может быть отрицательным! Установлено
0.\n");
graph[i][j] = 0;
}
else {
graph[i][j] = weight;

if (!isDirected) {
graph[j][i] = weight;
}
}
}
}
printf("Граф успешно введен.\n");
```

Далее выводится матрица смежности графа (рисунок 2):

Матрица смежности (неориентированный граф, 5 вершин):					
	0	1	2	3	4
0:	0	20	20	4	4
1:	20	0	30	5	5
2:	20	30	0	6	6
3:	4	5	6	0	2
4:	4	5	6	2	0

Рисунок 2 - матрица смежности графа

Если пользователь выбрал загрузку графа из файла (пункт 5), программа запрашивает имя файла и загружает данные:

```
char filename[256];
printf("Введите имя файла для загрузки: ");
scanf("%255s", filename);

FILE* f = fopen(filename, "r");
if (!f) {
    printf("Ошибка открытия файла %s!\n", filename);
    return;
}

int fileSize;
if (fscanf(f, "%d", &fileSize) != 1) {
    printf("Ошибка чтения размера графа из файла!\n");
    fclose(f);
    return;
}

if (fscanf(f, "%d", &isDirected) != 1) {
    printf("Ошибка чтения типа графа из файла!\n");
    fclose(f);
    return;
}
```

```

if (fileSize <= 0) {
printf("Некорректный размер графа в файле: %d\n", fileSize);
fclose(f);
return;
}
freeGraph();
graph = allocateGraph(fileSize);
if (graph == NULL) {
printf("Ошибка выделения памяти для графа!\n");
fclose(f);
return;
}

n = fileSize;
int error = 0;

for (int i = 0; i < n && !error; i++) {
for (int j = 0; j < n && !error; j++) {
if (fscanf(f, "%d", &graph[i][j]) != 1) {
printf("Ошибка чтения элемента [%d][%d] из файла!\n", i, j);
error = 1;
}
}
if (graph[i][j] < 0) {
printf("Отрицательный вес ребра [%d][%d] = %d. Исправлено на 0.\n",
i, j, graph[i][j]);
graph[i][j] = 0;
}
}
}

fclose(f);

if (!error) {

```

```

        printf("Граф успешно загружен из файла %s\n", filename);
printf("Тип: %s, Размер: %d вершин\n",
        isDirected ? "ориентированный" : "неориентированный", n);
    }
else {
    freeGraph();
}

```

После задания графа пользователь может выполнить алгоритм Дейкстры (пункт 8). Программа запрашивает начальную вершину, выполняется алгоритм Дейкстры и выводятся результаты (рисунок 3)

```

Доступные вершины: 0..4
Начальная вершина: 2

Стартовая вершина: 2
Тип графа: неориентированный

Кратчайшие расстояния:
до 0 = 10
до 1 = 11
до 2 = 0 (стартовая вершина)
до 3 = 6
до 4 = 6

Кратчайшие пути:
В 0 (10): 2 -> 3 -> 0
В 1 (11): 2 -> 3 -> 1
В 3 (6): 2 -> 3
В 4 (6): 2 -> 4

```

Рисунок 3 - результат выполнения алгоритма Дейкстры

Программа также предлагает сохранить результаты в файл (рисунок 4):

```

Введите имя файла для сохранения результатов (или нажмите Enter для пропуска): res.txt
Результаты сохранены в файл res.txt

```

Рисунок 4 - Сохранение результатов

5. Тестирование

Таблица 1 - Тестирование функциональности программы

№ теста	Описание теста	Проводимые действия	Ожидаемы результат
1	Запуск программы	Запуск программы, просмотр главного меню	Отображение меню с 8 пунктами
2	Задание размера графа	Выбор п.1, ввод: 5 вершин	Установка размера графа = 5
3	Задание типа графа	Выбор п.2, ввод: 0 (неориентированный)	Установка типа графа = неориентированный
4	Ручной ввод графа	Выбор п.3, ввод матрицы 5x5	Успешный ввод матрицы смежности
5	Генерация случайного графа	Выбор п.4, параметры: max=10, density=50%	Успешная генерация графа
6	Загрузка из файла	Выбор п.5, имя файла: res.txt	Успешная загрузка графа из файла
7	Просмотр матрицы	Выбор п.7	Отображение матрицы
8	Выполнение алгоритма	Выбор п.8, стартовая вершина: 0	Корректный расчёт кратчайших путей
9	Сохранение результатов	После алгоритма, ввод имени файла	Сохранение результатов в файл
10	Выход из программы	Выбор п.0	Корректное завершение программы

Среда разработки Microsoft Visual Studio предоставляет все средства, необходимые при разработке и отладке многомодульной программы.

Тестирование проводилось в рабочем порядке, в процессе разработки, после завершения написания программы. В ходе тестирования было выявлено и исправлено множество проблем, связанных с вводом данных, изменением дизайна выводимых данных, алгоритмом программы, взаимодействием функций.

Ниже продемонстрирован результат тестирования программы при вводе пользователем различных графов и выполнении алгоритма Дейкстры.

Матрица смежности (ориентированный граф, 5 вершин):

	0	1	2	3	4
0:	0	0	0	1	8
1:	0	0	0	1	4
2:	8	0	0	0	0
3:	0	0	4	0	0
4:	0	5	6	9	0

Рисунок 5 - Ориентированный граф с 5 вершинами

```
Доступные вершины: 0..4
Начальная вершина: 0

Стартовая вершина: 0
Тип графа: ориентированный

Кратчайшие расстояния:
до 0 = 0 (стартовая вершина)
до 1 = 13
до 2 = 5
до 3 = 1
до 4 = 8

Кратчайшие пути:
В 1 (13): 0 -> 4 -> 1
В 2 (5): 0 -> 3 -> 2
В 3 (1): 0 -> 3
В 4 (8): 0 -> 4
```

Рисунок 6 - результат выполнения алгоритма с ориентированным графом

Матрица смежности (неориентированный граф, 6 вершин):						
	0	1	2	3	4	5
0:	0	6	4	0	0	0
1:	6	0	1	5	0	8
2:	4	1	0	6	0	10
3:	0	5	6	0	0	0
4:	0	0	0	0	0	3
5:	0	8	10	0	3	0

Рисунок 7 - Неориентированный граф с 6 вершинами

```

Доступные вершины: 0..5
Начальная вершина: 0

Стартовая вершина: 0
Тип графа: неориентированный

Кратчайшие расстояния:
до 0 = 0 (стартовая вершина)
до 1 = 5
до 2 = 4
до 3 = 10
до 4 = 16
до 5 = 13

Кратчайшие пути:
В 1 (5): 0 -> 2 -> 1
В 2 (4): 0 -> 2
В 3 (10): 0 -> 2 -> 3
В 4 (16): 0 -> 2 -> 1 -> 5 -> 4
В 5 (13): 0 -> 2 -> 1 -> 5

```

Рисунок 8 - результат выполнения алгоритма с неориентированным графом

```

Введите матрицу смежности (2x2):
(0 – нет ребра, >0 – вес ребра)
graph[0][1]: -3
Вес не может быть отрицательным! Установлено 0.

```

Рисунок 9 - Проверка на отрицательные веса

В таблице 2 систематизированы результаты проверенного тестирования.

Таблица 2 - Результат функциональности программы

№ теста	Описание теста	Фактический результат
1	Запуск программы	Верно
2	Задание размера графа	Верно
3	Задание типа графа	Верно
4	Ручной ввод графа	Верно
5	Генерация случайного графа	Верно
6	Загрузка из файла	Верно (при наличии файла)
7	Просмотр матрицы	Верно
8	Выполнение алгоритма	Верно
9	Сохранение результатов	Верно
10	Выход из программы	Верно

В результате тестирования было выявлено, что программа успешно проверяет данные на соответствие необходимым требованиям.

6. Ручной расчёт задачи

Проведём проверку программы посредством ручных вычислений на примере взвешенного графа с 5 вершинами. Рассмотрим следующий неориентированный граф (рисунок 10):

Матрица смежности (неориентированный граф, 5 вершин):					
	0	1	2	3	4
0:	0	1	31	5	46
1:	1	0	28	0	0
2:	31	28	0	4	23
3:	5	0	4	0	41
4:	46	0	23	41	0

Рисунок 10 - неориентированный граф, 5 вершин

Начинаем расчёт из вершины 0. Проверяем, есть ли путь из 0 в другие вершины. В нашем случае есть пути в вершины 1 (вес 1), 2 (вес 31), 3 (вес 5) и 4 (вес 46).

Из всех непосещенных вершин выбираем вершину с минимальным расстоянием. Это вершина 1 с расстоянием 1. Помечаем вершину 1 как посещенную.

Из вершины 1 проверяем пути к соседним вершинам. Есть путь в вершину 2 (вес 28). До вершины 2 было расстояние 31, а через вершину 1 получается $1 + 28 = 29$. Это меньше, поэтому обновляем расстояние до вершины 2 на 29.

Теперь из непосещенных вершин минимальное расстояние у вершины 3 - 5. Помечаем вершину 3 как посещенную.

Из вершины 3 проверяем пути к соседним вершинам. Есть путь в вершину 2 (вес 4) и в вершину 4 (вес 41). До вершины 2 было расстояние 29, а через вершину 3 получается $5 + 4 = 9$. Это меньше, поэтому обновляем расстояние до вершины 2 на 9. До вершины 4 было расстояние 46, а через вершину 3 получается $5 + 41 = 46$. Расстояние не изменилось.

Теперь из непосещенных вершин минимальное расстояние у вершины 2 - 9. Помечаем вершину 2 как посещенную.

Из вершины 2 проверяем пути к соседним вершинам. Есть путь в вершину 4 (вес 23). До вершины 4 было расстояние 46, а через вершину 2 получается $9 + 23 = 32$. Это меньше, поэтому обновляем расстояние до вершины 4 на 32.

Теперь из непосещенных вершин минимальное расстояние у вершины 4 - 32. Помечаем вершину 4 как посещенную.

Все вершины посещены, алгоритм завершен. В итоге получили следующие кратчайшие расстояния от вершины 0:

$\text{dist}[0] = 0, \text{dist}[1] = 1, \text{dist}[2] = 9, \text{dist}[3] = 5, \text{dist}[4] = 32$.

Для восстановления пути используем информацию о предыдущих вершинах:

Путь в вершину 1: $0 \rightarrow 1$ (расстояние 1), путь в вершину 2: $0 \rightarrow 3 \rightarrow 2$ (расстояние 9), путь в вершину 3: $0 \rightarrow 3$ (расстояние 5), Путь в вершину 4: $0 \rightarrow 3 \rightarrow 2 \rightarrow 4$ (расстояние 32).

Таким образом, мы вручную рассчитали кратчайшие пути от вершины 0 до всех остальных вершин графа. Результат работы приложения соответствует ручным расчетам, что свидетельствует о правильной работе (рисунок 11).

```
Доступные вершины: 0..4
Начальная вершина:
0

Стартовая вершина: 0
Тип графа: неориентированный

Кратчайшие расстояния:
до 0 = 0 (стартовая вершина)
до 1 = 1
до 2 = 9
до 3 = 5
до 4 = 32

Кратчайшие пути:
В 1 (1): 0 -> 1
В 2 (9): 0 -> 3 -> 2
В 3 (5): 0 -> 3
В 4 (32): 0 -> 3 -> 2 -> 4
```

Рисунок 11 - Тестирование работы программы

Заключение

Таким образом, в процессе создания данного проекта разработана программа, реализующая алгоритм Дейкстры для поиска кратчайших путей во взвешенных графах в Microsoft Visual Studio.

При выполнении данной курсовой работы были получены навыки разработки программ и освоены приёмы создания матриц смежности, основанных на теории графов. Приобретены навыки по осуществлению алгоритма Дейкстры для поиска кратчайших путей. Углублены знания языка программирования Си, включая работу с динамическими структурами данных, файловым вводом-выводом и обработкой ошибок.

Недостатком разработанной программы является примитивный пользовательский интерфейс. Программа работает в консольном режиме, что не добавляет к сложности языка сложность программного оконного интерфейса. Программа имеет небольшой, но достаточный для использования функционал возможностей.

Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: Построение и анализ - М.: МЦНМО, 2001. - 960 с.
2. Кристофидес Н. «Теория графов. Алгоритмический подход» - Мир, 1978
3. Герберт Шилдт «Полный справочник по C++» - Вильямс, 2006
4. Уилсон Р. Введение в теорию графов. Пер. с англ. 1977. 208 с.
5. Харви Дейтел, Пол Дейтел. Как программировать на C/C++. 2009 г.
6. З. Оре О. Графы и их применение: Пер. с англ. 1965. 176 с.

Приложение А. Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <locale.h>
#include <conio.h>
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>
#include <string.h>

int** graph = NULL;
int n = 0;
int isDirected = 0;

void freeGraph() {
    if (graph != NULL) {
        for (int i = 0; i < n; i++) {
            free(graph[i]);
        }
        free(graph);
        graph = NULL;
    }
    n = 0;
}

int** allocateGraph(int size) {
    int** newGraph = (int**)malloc(size * sizeof(int*));
    if (newGraph == NULL) return NULL;

    for (int i = 0; i < size; i++) {
        newGraph[i] = (int*)malloc(size * sizeof(int));
        if (newGraph[i] == NULL) {
            for (int j = 0; j < i; j++) free(newGraph[j]);
        }
    }
}
```

```

        free(newGraph);
        return NULL;
    }
}
return newGraph;
}

void initGraph(int size) {
    freeGraph();

    graph = allocateGraph(size);
    if (graph == NULL) {
printf("Ошибка выделения памяти!\n");
n = 0;
        return;
    }

    n = size;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                graph[i][j] = 0;
            else
                graph[i][j] = 0;
        }
    }
}

int inputInt(const char* prompt) {
    int value;
    printf("%s", prompt);
    while (scanf("%d", &value) != 1) {
        printf("Ошибка! Введите целое число: ");
        while (getchar() != '\n');
    }
}

```

```

    }
    return value;
}

int inputIntRange(const char* prompt, int min, int max) {
    int value;
    do {
        value = inputInt(prompt);
        if (value < min || value > max) {
printf("Ошибка! Введите число от %d до %d: ", min, max);
        }
    } while (value < min || value > max);
    return value;
}

void printGraph() {
    if (n == 0 || graph == NULL) {
printf("Граф не задан!\n");
        return;
    }

    printf("\nМатрица смежности (%sграф, %d вершин):\n",
        isDirected ? "ориентированный " : "неориентированный ", n);
printf("");
    for (int i = 0; i < n; i++)
        printf("%4d ", i);
printf("\n");

    for (int i = 0; i < n; i++) {
        printf("%4d: ", i);
        for (int j = 0; j < n; j++) {
            printf("%4d ", graph[i][j]);
        }
        printf("\n");
    }
}

```

```

    }
}

void inputManual() {
    if (n == 0) {
printf("Сначала задайте размер графа (пункт 1)!\n");
        return;
    }

    printf("Введите матрицу смежности (%dx%d):\n", n, n);
    printf("(0 - нет ребра, >0 - вес ребра)\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) {
                graph[i][j] = 0;
                continue;
            }

            printf("graph[%d][%d]: ", i, j);
            int weight;
            scanf("%d", &weight);

            if (weight < 0) {
                printf("Вес не может быть отрицательным! Установлено
0.\n");
                graph[i][j] = 0;
            }

            else {
                graph[i][j] = weight;

                if (!isDirected) {
                    graph[j][i] = weight;
                }
            }
        }
    }
}

```

```

        }
    }
}
printf("Граф успешно введен.\n");
}

void inputRandom() {
    if (n == 0) {
        printf("Сначала задайте размер графа (пункт 1)!\n");
        return;
    }

    int maxWeight = inputIntRange("Максимальный вес ребра (1-100): ", 1,
100);
    int density = inputIntRange("Плотность графа в % (0-100): ", 0, 100);

    srand((unsigned int)time(NULL));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) {
                graph[i][j] = 0;
                continue;
            }

            if ((rand() % 100) < density) {
                int weight = rand() % maxWeight + 1;
                graph[i][j] = weight;

                if (!isDirected) {
                    graph[j][i] = weight;
                }
            }
            else {

```

```

        graph[i][j] = 0;
        if (!isDirected) {
            graph[j][i] = 0;
        }
    }
}

printf("Случайный      %sграфсгенерирован.\n",      isDirected      ?
"ориентированный" : "неориентированный");
}

void inputFromFile() {
    char filename[256];
    printf("Введите имя файла для загрузки: ");
    scanf("%255s", filename);

    FILE* f = fopen(filename, "r");
    if (!f) {
        printf("Ошибкаоткрытияфайла %s!\n", filename);
        return;
    }

    int fileSize;
    if (fscanf(f, "%d", &fileSize) != 1) {
        printf("Ошибка чтения размера графа из файла!\n");
        fclose(f);
        return;
    }

    if (fscanf(f, "%d", &isDirected) != 1) {
        printf("Ошибка чтения типа графа из файла!\n");
        fclose(f);
        return;
    }
}

```

```

        if (fileSize <= 0) {
printf("Некорректный размер графа в файле: %d\n", fileSize);
fclose(f);
        return;
    }

    freeGraph();
    graph = allocateGraph(fileSize);
    if (graph == NULL) {
        printf("Ошибка выделения памяти для графа!\n");
fclose(f);
        return;
    }

    n = fileSize;
    int error = 0;

    for (int i = 0; i < n && !error; i++) {
        for (int j = 0; j < n && !error; j++) {
            if (fscanf(f, "%d", &graph[i][j]) != 1) {
printf("Ошибка чтения элемента [%d][%d] из файла!\n", i, j);
error = 1;
            }
            if (graph[i][j] < 0) {
printf("Отрицательный вес ребра [%d][%d] = %d. Исправлено на 0.\n",
i, j, graph[i][j]);
                graph[i][j] = 0;
            }
        }
    }

    fclose(f);

```

```

        if (!error) {
            printf("Граф успешно загружен из файла %s\n", filename);
printf("Тип: %s, Размер: %d вершин\n",
            isDirected ? "ориентированный" : "неориентированный", n);
        }
        else {
            freeGraph();
        }
    }

void exportGraph() {
    if (n == 0 || graph == NULL) {
printf("Граф не задан!\n");
        return;
    }

    char filename[256];
    printf("Введите имя файла для экспорта: ");
    scanf("%255s", filename);

    FILE* f = fopen(filename, "w");
    if (!f) {
        printf("Ошибка создания файла %s!\n", filename);
        return;
    }

    fprintf(f, "%d\n", n);
    fprintf(f, "%d\n", isDirected);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            fprintf(f, "%d ", graph[i][j]);
        }
        fprintf(f, "\n");
    }
}

```

```

    }

    fclose(f);
    printf("Граф экспортирован в файл %s\n", filename);
}

void printPath(int prev[], int destination) {
    if (prev[destination] == -1) {
        printf("%d", destination);
        return;
    }

    int* path = (int*)malloc(n * sizeof(int));
    int count = 0;
    int current = destination;

    while (current != -1) {
        path[count++] = current;
        current = prev[current];
    }

    for (int i = count - 1; i >= 0; i--) {
        printf("%d", path[i]);
        if (i > 0) printf(" ->");
    }

    free(path);
}

void dijkstra(int start) {
    if (n == 0 || graph == NULL) {
        printf("Графнезадан!\n");
        return;
    }
}

```

```

        if (start < 0 || start >= n) {
printf("Неверная стартовая вершина! Допустимый диапазон: 0..%d\n", n -
1);
return;
        }

        int* dist = (int*)malloc(n * sizeof(int));
        int* visited = (int*)malloc(n * sizeof(int));
        int* prev = (int*)malloc(n * sizeof(int));

        if (dist == NULL || visited == NULL || prev == NULL) {
printf("Ошибка выделения памяти!\n");
if (dist) free(dist);
        if (visited) free(visited);
        if (prev) free(prev);
        return;
        }

        int hasNegativeEdge = 0;
        for (int i = 0; i < n && !hasNegativeEdge; i++) {
            for (int j = 0; j < n && !hasNegativeEdge; j++) {
if (graph[i][j] < 0) {
                printf("Ошибка: найден отрицательный вес ребра %d->%d
= %d\n",
i, j, graph[i][j]);
                hasNegativeEdge = 1;
            }
        }
    }

    if (hasNegativeEdge) {
        printf("Алгоритм Дейкстры не работает с отрицательными весами!\n");
        free(dist);
    }

```

```

    free(visited);
    free(prev);
    return;
}

for (int i = 0; i < n; i++) {
    dist[i] = INT_MAX;
    visited[i] = 0;
    prev[i] = -1;
}

dist[start] = 0;

for (int count = 0; count < n; count++) {
    int u = -1;
    int minDist = INT_MAX;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] < minDist) {
            minDist = dist[i];
            u = i;
        }
    }

    if (u == -1 || minDist == INT_MAX) {
        break;
    }

    visited[u] = 1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && graph[u][v] > 0) {
            long long newDist = (long long)dist[u] + graph[u][v];
            if (newDist < dist[v]) {

```

```

        dist[v] = (int)newDist;
        prev[v] = u;
    }
}

}

printf("\nСтартоваявершина: %d\n", start);
printf("Типграфа:  %s\n",  isDirected  ?  "ориентированный"  :
"неориентированный");

printf("\nКратчайшие расстояния:\n");
for (int i = 0; i < n; i++) {
    if (i == start) {
        printf("до %d = 0 (стартоваявершина)\n", i);
    }
    else if (dist[i] == INT_MAX) {
printf("до %d = недостижима\n", i);
    }
    else {
        printf("до %d = %d\n", i, dist[i]);
    }
}

printf("\nКратчайшиепути:\n");
for (int i = 0; i < n; i++) {
    if (i != start && dist[i] != INT_MAX) {
printf("В %d (%d): ", i, dist[i]);
        printPath(prev, i);
        printf("\n");
    }
}

char filename[256];

```

```

    printf("\nВведите имя файла для сохранения результатов (или
нажмите Enter для пропуска): ");
    getchar();
    fgets(filename, sizeof(filename), stdin);

    filename[strcspn(filename, "\n")] = 0;

    if (strlen(filename) > 0) {
        FILE* f = fopen(filename, "w");
    if (f) {
        fprintf(f, "Исходные данные:\n");
        fprintf(f, "Стартовая вершина: %d\n", start);
        fprintf(f, "Тип графа: %s\n", isDirected ? "ориентированный" :
"неориентированный");
        fprintf(f, "Количество вершин: %d\n\n", n);

        fprintf(f, "Исходная матрица смежности:\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                fprintf(f, "%d ", graph[i][j]);
            }
            fprintf(f, "\n");
        }

        fprintf(f, "Кратчайшие расстояния от вершины %d:\n", start);
        for (int i = 0; i < n; i++) {
            if (i == start) {
                fprintf(f, "до %d = 0 (стартовая вершина)\n", i);
            }
            else if (dist[i] == INT_MAX) {
                fprintf(f, "до %d = недостижима\n", i);
            }
            else {
                fprintf(f, "до %d = %d\n", i, dist[i]);
            }
        }
    }
}

```

```

    }

    }

    fprintf(f, "\nКратчайшие пути от вершины %d:\n", start);
    for (int i = 0; i < n; i++) {
        if (i != start && dist[i] != INT_MAX) {
            fprintf(f, "В %d (расстояние: %d): ", i, dist[i]);

            int* path = (int*)malloc(n * sizeof(int));
            int count = 0;
            int current = i;
            while (current != -1) {
                path[count++] = current;
                current = prev[current];
            }

            for (int j = count - 1; j >= 0; j--) {
                fprintf(f, "%d", path[j]);
                if (j > 0) fprintf(f, " ->");
            }
            fprintf(f, "\n");

            free(path);
        }
    }

    fclose(f);
    printf("Результаты сохранены в файл %s\n", filename);
}
else {
    printf("Ошибка сохранения в файл %s!\n", filename);
}
}

```

```

    free(dist);
    free(visited);
    free(prev);

    printf("\nНажмите любую клавишу для продолжения...");
    _getch();
}

int main() {
    setlocale(LC_ALL, "Russian");

    int choice;
    while (1) {
        printf("1. Задатьразмерграфа (текущий: %d)\n", n);
        printf("2. Задать тип графа (текущий: %s)\n",
            isDirected ? "ориентированный" : "неориентированный");
        printf("3. Ввод графа вручную\n");
        printf("4. Генерация случайного графа\n");
        printf("5. Загрузить граф из файла\n");
        printf("6. Экспорт графа в файл\n");
        printf("7. Показать матрицу смежности\n");
        printf("8. Выполнить алгоритм Дейкстры\n");
        printf("0. Выход\n");

        choice = inputIntRange("Выбор: ", 0, 8);

        if (choice == 0) {
            break;
        }

        switch (choice) {
            case 1: {
                int size = inputInt("Количествовершин: ");
                if (size <= 0) {

```

```

printf("Размер должен быть положительным!\n");
    break;
}
initGraph(size);
printf("Размер графа установлен в %d вершин.\n", n);
break;
}

case 2:
    isDirected = inputIntRange("Тип графа (0 -
неориентированный, 1 - ориентированный): ", 0, 1);
    printf("Тип графа установлен: %s\n",
        isDirected ? "ориентированный" : "неориентированный");
break;

case 3:
    inputManual();
    break;

case 4:
    inputRandom();
    break;

case 5:
    inputFromFile();
    break;

case 6:
    exportGraph();
    break;

case 7:
    printGraph();
    break;

```

```

        case 8: {
            if (n == 0) {
                printf("Сначала задайте граф!\n");
break;

                }
                printf("Доступные вершины: 0..%d\n", n - 1);
int start = inputIntRange("Начальная вершина: ", 0, n - 1);
                dijkstra(start);
                break;
            }

            default:
                printf("Неверный выбор!\n");
            }
        }

        freeGraph();

        printf("\nПрограмма завершена. Нажмите любую клавишу...");
_getch();
return

```

